SurgeAI • Control Tower

Illustrated Technical Deep-Dive with Upgrade Roadmap

This revision adds inline architecture and flow diagrams to every section, plus an **Innovation Opportunity** callout under every major sub-point. Diagrams are drawn with vector primitives (not raster images) so they stay crisp at any zoom, and every claim remains anchored to a file:line reference in the repository.

Document rev	Rev B — illustrated + innovation callouts
Domain	Retail / Supply-Chain Demand & Risk Analytics
Backend	FastAPI 0.109 + Uvicorn + Pydantic v2
Frontend	React 19 + TypeScript strict + Vite 8 + Tailwind 3
ML stack	XGBoost, LightGBM, Random Forest, multi-target regressors
Deployment	Vercel (static SPA) + self-hosted Uvicorn (ASGI API)
Diagrams	System, Request Lifecycle, ML Pipeline, Frontend Layers, Deployment, Scale-Out, Sequence

1. System Design

SurgeAI is a **three-tier composable pipeline**: (a) an offline ML factory that emits artifacts to disk, (b) a stateless FastAPI runtime that loads those artifacts once at startup, and (c) a code-split React 19 dashboard that consumes JSON over HTTP. No database is in the critical path.

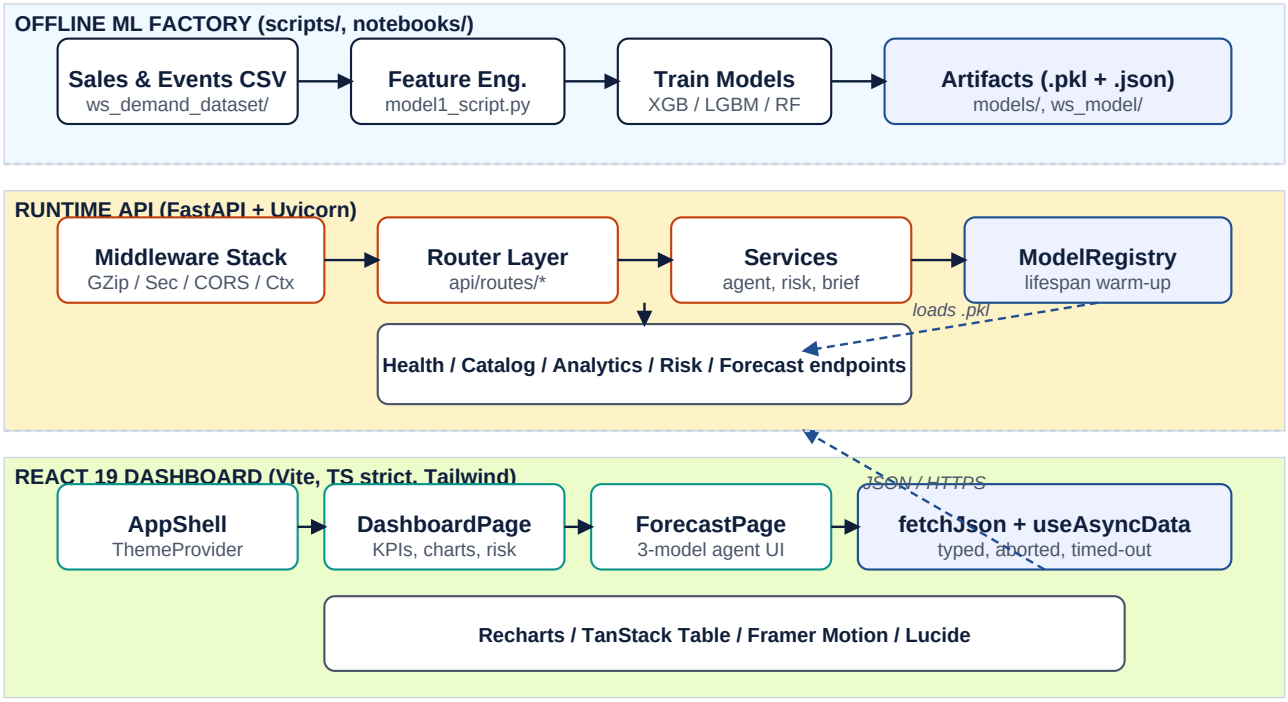


Figure 1.1 — Three-tier system architecture (offline factory → runtime API → dashboard).

1.1 Bounded Contexts

The backend is split into four bounded contexts, each in its own package: *core/* for cross-cutting infra (logging, middleware, ModelRegistry), *services/* for pure business logic, *api/* for HTTP adapters, and *schemas.py* + *config.py* for the type contract with the outside world.

Code ref → `backend/app/main.py:1-116` • `backend/app/api/router.py:1-19`

> ■ **INNOVATION OPPORTUNITY — Bounded Contexts**
>
> ■ Introduce a **domain-events bus** (e.g. Redis Streams or NATS) so services publish “risk_flagged”, “forecast_generated” events consumed by future workers (notifications, audit, retraining).
>
> ■ Extract *services/* into a proper Python package with its own tests and semantic version — makes it drop-in for a batch/CLI worker.
>
> ■ Add **hexagonal ports/adapters** — right now services read CSV directly; a small port interface would let Snowflake/Postgres slot in with zero service changes.

1.2 Startup / Lifespan Wiring

FastAPI's *lifespan* context manager runs a single, well-defined warm-up: every model is loaded exactly once from disk into a process-global *ModelRegistry*. If an artifact is missing the process still starts, the health endpoint downgrades to yellow, and the frontend renders a “setup required” screen instead of 5xx-ing.

Code ref → `backend/app/main.py:39-70` • `backend/app/core/state.py:14-61`

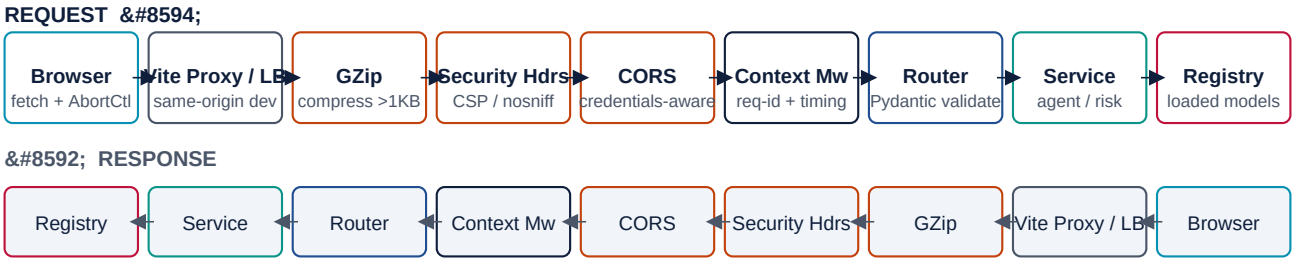
■ INNOVATION OPPORTUNITY — Startup / Lifespan

■ Add a **warm-up prediction** at lifespan-end (dummy features → predict) so JIT/first-call latency is amortised before traffic arrives.
■ Load models from a signed **OCI artifact registry** instead of the container image, so retraining does not force a new image build.
■ Add a **model version manifest** (SHA + trained-at) exposed on /api/health so canary rollouts can verify propagation before flipping traffic.

1.3 Request Lifecycle

Every request passes bottom-up through GZip → Security Headers → CORS → Request-Context. The context middleware stamps a 12-char correlation id, propagates it via *ContextVar*, logs status + duration, and emits *Server-Timing* so the browser's network panel doubles as APM.

Request lifecycle — every hop is instrumented and typed



Response: JSON + X-Request-ID + Server-Timing headers

Figure 1.2 — Request lifecycle. Every hop is typed, timed, and correlation-tagged.

Code ref → `backend/app/main.py:88-104 • backend/app/core/middleware.py:24-104`

■ INNOVATION OPPORTUNITY — Request Lifecycle

■ Replace manual middleware with **OpenTelemetry auto-instrumentation** — traces get exported to Tempo/Jaeger with zero application code changes.
■ Add a **per-endpoint rate limiter** (*slowapi*) — /api/demand/forecast is expensive; protect it explicitly.
■ Emit **Prometheus histograms** (`http_request_duration_seconds{route, code}`) from the context middleware so p50/p95/p99 SLOs become measurable.

1.4 Stateless Data Plane

The service holds only immutable loaded models plus per-request DataFrames materialised from CSV via *tables.py*. Reads never mutate; writes never happen. This removes a whole class of consistency problems and makes replicas interchangeable.

Code ref → `backend/app/services/tables.py:1-69 • backend/app/api/deps.py:1-19`

■ **INNOVATION OPPORTUNITY — Data Plane**

■ Wrap *load_demand_predictions* in a **LRU cache keyed by file mtime** — removes the last hot-path I/O with zero staleness risk.
■ Migrate the analytics tables to **Parquet + DuckDB** or **Polars**: 10-30× faster reads and pushdown filtering on date ranges.
■ Introduce a **materialised-view refresh job** that pre-computes the risk register nightly — the request path becomes a single row read.

1.5 Graceful Degradation

Failures never crash the service. A missing CSV logs and returns None; a missing model raises 503 with the exact remediation script; a corrupt row is reported via /api/health.messages. The health schema enumerates every artifact's presence flag.

Code ref → `backend/app/api/routes/system.py:14-43 • backend/app/schemas.py:6-16`

■ **INNOVATION OPPORTUNITY — Graceful Degradation**

■ Add a **circuit-breaker** (*purgatory* or *pybreaker*) around each model — if a model start throwing, degrade the composite to baseline-only and surface it in *insights[]*.
■ Publish a **Ready vs Live** split: /api/health/live (process alive) vs /api/health/ready (artifacts loaded) — Kubernetes probes get precise semantics.
■ Ship a **chaos-test** script that deletes an artifact then hits every endpoint — proves the setup screen truly works.

1.6 Frontend Composition

React mirrors the backend's split: *pages/* are the route entries, *features/* are business widgets, *components/ui/* is the design-system primitives layer. Pages are lazy-loaded with React.lazy and wrapped in per-section *ErrorBoundary*.

Frontend layer diagram — atomic composition

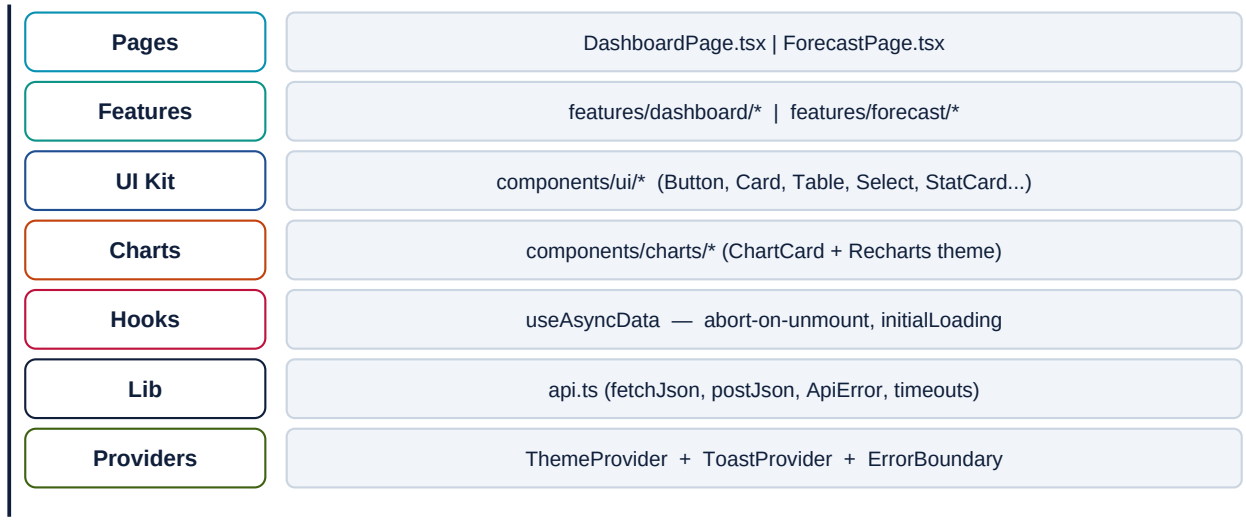


Figure 1.3 — Frontend atomic composition. Higher rows depend downward only.

Code ref → `frontend/src/App.tsx:1-77 • frontend/src/main.tsx:1-24`

■ INNOVATION OPPORTUNITY — Frontend Composition

■ Migrate to **TanStack Router** — brings type-safe routes and preload-on-hover, cutting perceived latency between Dashboard and Forecast.

■ Publish the *components/ui/* layer as a versioned internal **Storybook** + npm package — makes cross-app reuse trivial.

■ Add **React Server Components** (Next.js migration) for the initial KPI shell, so the first paint is HTML with data already inlined.

2. Machine Learning

SurgeAI's intelligence lives in a **3-model ensemble** aggregated at inference time by a lightweight agent. Each model attacks one dimension of demand variability the others cannot explain, is trained offline, and ships as a joblib pickle plus a JSON feature manifest so inference rebuilds the exact schema.

3-Model composite forecast pipeline

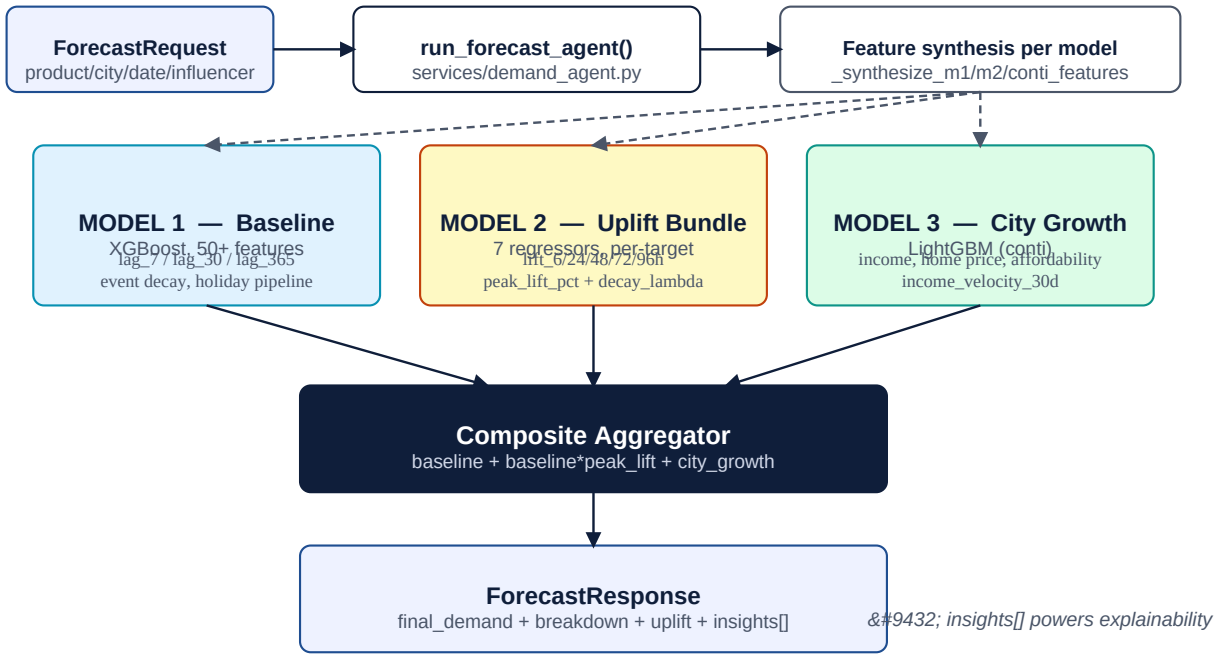


Figure 2.1 — 3-model composite forecast pipeline with feature-synthesis fan-out.

2.1 Model 1 — Baseline Demand (XGBoost)

Trained on a time-based split (train < 2024-10-01, test on Q4) using **WMAPE** as the primary metric because high-volume SKUs contribute more to error and to lost revenue. Features are grouped into seven families: lag/rolling, time, event-decay (11 events), interaction terms, segment, registry signal, and product-identity encodings. The event-decay math peaks precisely on the event day, post-hangover uses a separate tau.

Code ref → `scripts/model1_script.py:454-666 (features) • 938-1055 (training + WMAPE)`

■ INNOVATION OPPORTUNITY — Model 1 — Baseline Demand

■ Swap XGBoost for a **global hierarchical model** (Temporal Fusion Transformer / N-BEATSx / PatchTST) — captures cross-SKU sharing and gives probabilistic quantile forecasts, not point predictions.

■ Add **Prophet-style holiday exogenous regressors** alongside the manual decay features — reduces the >600-line feature-engineering script by ~40%.

■ Introduce **conformal prediction intervals** so the UI shows P10/P50/P90 bands instead of a single point — buyers understand risk, not just mean demand.

2.2 Model 2 — Influencer Uplift Bundle

Not one model but a **bundle of seven independently-trained regressors**, one per target: *lift_6/24/48/72/96h*, *peak_lift_pct*, *decay_lambda*. Loaded lazily — a missing pickle downgrades the

target rather than breaking the API. This is real MLOps: independent versioning per target so 24h lift retrains without disturbing 96h tail.

Code ref → `backend/app/services/demand_agent.py:66-176` • `models/lift_*_best.pkl`

> ■ **INNOVATION OPPORTUNITY — Model 2 — Influencer Uplift**
>
> ■ Replace the seven per-horizon regressors with **one temporal decay network** parameterised on (peak_lift, lambda) — training set becomes richer, params drop from ~7× to 1×.
>
> ■ Add **causal uplift modelling** (T-Learner or X-Learner from EconML) — separates the campaign effect from baseline drift, giving true incremental lift.
>
> ■ Enrich features with **influencer graph embeddings** (creator-audience-brand tripartite) — captures why 2 influencers with identical follower counts drive very different lift.

2.3 Model 3 — City Growth (LightGBM)

Layers macroeconomic context onto the SKU forecast: median_income, home_price, affordability_ratio, income_velocity_30d, price_velocity_30d, rolling_units_30d, plus one-hot city dummies. Output is an additive units correction. LightGBM captures non-linear wealth × luxury-sensitivity interactions.

Code ref → `backend/app/services/demand_agent.py:11-19, 88-119, 180-193` • `notebooks/conti.ipynb`

> ■ **INNOVATION OPPORTUNITY — Model 3 — City Growth**
>
> ■ Replace one-hot city dummies with a **learned city embedding** (Entity Embeddings) — handles unseen cities gracefully.
>
> ■ Pull **live macro feeds** (World Bank / OECD / RBI APIs) via a nightly ETL — the model then reflects real income/price velocity, not synthetic profiles.
>
> ■ Add **geospatial features** (population density, competitor density, tourism index) via H3 hex indexing — captures within-country demand heterogeneity.

2.4 Composite Forecast Agent

run_forecast_agent() is a 90-line procedural function that: (1) synthesises Model 1 features, (2) predicts baseline, (3) conditionally invokes Model 2 only when a campaign is active, (4) predicts city growth, (5) returns a typed response with a human-readable *insights* list.

Code ref → `backend/app/services/demand_agent.py:121-210` • `backend/app/api/routes/forecast.py:43-58`

> ■ **INNOVATION OPPORTUNITY — Composite Agent**
>
> ■ Turn *insights[]* into an **LLM-generated narrative** (Claude / GPT-4) that explains the composite in plain English — “Peak lift on day 3 driven by mid-tier Instagram creator + weekend + Q4 seasonality”.
>
> ■ Add **Bayesian model averaging** instead of a linear sum — learn per-context weights (city, category, event) so the composite is not fixed at compile-time.
>
> ■ Expose an **agent trace** in the response (each model's raw output, its feature dump, timings) — powers a debug panel for ML engineers.

2.5 Risk Engine — Rule-Based, ML-Adjacent

compute_risks reads the demand DataFrame (actuals + predictions) and inventory, emits severity-ranked stockout and overstock flags. Stockout uses **max(actual, predicted) run-rate** for conservatism; overstock triggers only when weeks-of-cover > 10 **and** trend is falling.

Code ref → `backend/app/services/risk_engine.py:22-125`

■ INNOVATION OPPORTUNITY — Risk Engine

■ Replace fixed thresholds with a **learned classifier** (LightGBM binary) trained on historical stockouts — captures SKU-specific risk profiles the rule set cannot.

■ Add **lead-time uncertainty** from supplier history (log-normal) and use **Monte Carlo** to produce P(stockout) rather than a boolean flag.

■ Add **action recommendations**: instead of “stockout risk”, emit “expedite PO-4471 or re-route 42 units from W2” — the register becomes a to-do list.

2.6 Serving Contract & Explainability

Predictions are exposed via three surfaces: POST /api/demand/forecast, GET /api/demand/timeseries, and GET /api/influencer/summary. Every response is a Pydantic model — mis-typed data cannot leave the process.

Code ref → `backend/app/schemas.py:47-83` • `backend/app/api/routes/forecast.py:19-58`

■ INNOVATION OPPORTUNITY — Serving Contract

■ Attach **SHAP values** to each forecast response — the top 5 contributing features per prediction render as a mini waterfall chart on the Forecast page.

■ Publish an **OpenAPI-derived Python + TypeScript SDK** (via *openapi-generator*) so downstream apps consume the API type-safely.

■ Add a **/api/models/registry** endpoint listing loaded models, their versions and training dates — makes A/B and shadow-mode deploys observable.

3. Development Technologies

The stack was chosen to minimise the surface area a single engineer must master while remaining credibly production-grade. Every dependency earns its place.

Frontend layer diagram — atomic composition

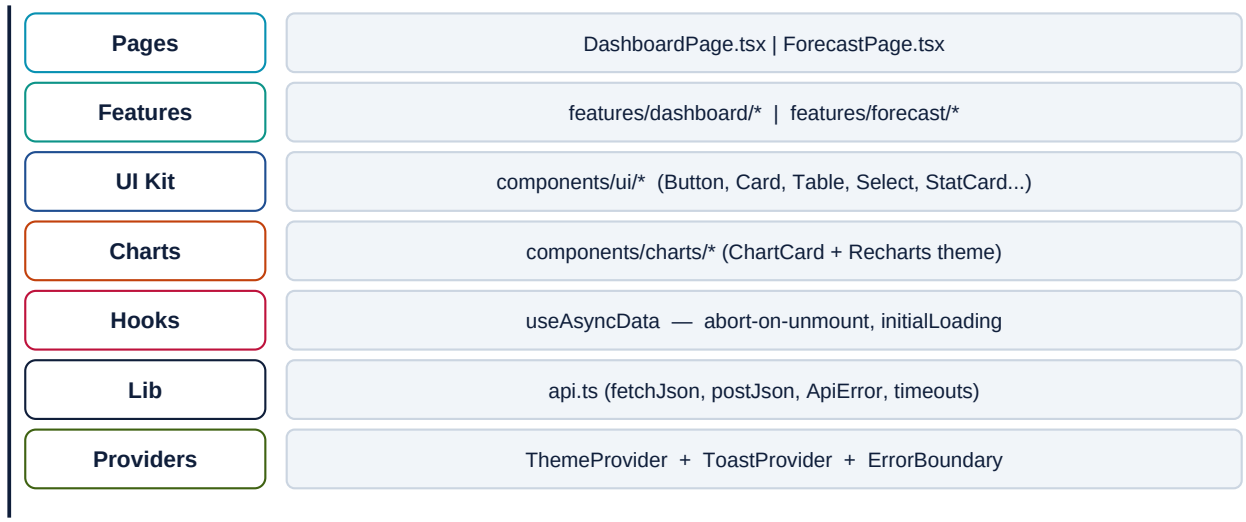


Figure 3.1 — Frontend layer stack. Providers/Lib at the base, Pages at the top.

3.1 Backend — Python 3.10 + FastAPI 0.109

FastAPI + Uvicorn[standard] + Pydantic v2 + pydantic-settings. Data with pandas 2 + NumPy, models via joblib + scikit-learn + xgboost 2 + lightgbm 4. Version bounds are pinned as ranges (>=0.109,<1) so security patches land automatically while majors do not surprise us.

Code ref → `backend/requirements.txt • backend/app/config.py:23-113`

■ INNOVATION OPPORTUNITY — Backend Stack

■ Adopt **uv** or **Poetry** for lock-file-based reproducible installs — *requirements.txt* ranges are convenient but not deterministic.

■ Move to **Python 3.12**: faster startup, per-interpreter GIL primitives, better error messages — no code changes required.

■ Wrap CPU-bound predict calls in **asyncio.to_thread** so a slow model does not stall the event loop under concurrent load.

3.2 Frontend — React 19 + TypeScript strict + Vite 8

React 19 concurrent renderer, TS strict with typed API responses, Vite 8 with a /api proxy for same-origin dev, Tailwind 3 as the token layer for light/dark/system, Recharts 3 for viz, TanStack Table 8 for the risk register, framer-motion for micro-interactions, ESLint 9 flat config with jsx-a11y so a11y is enforced at CI.

Code ref → `frontend/package.json:12-39 • frontend/vite.config.ts:1-52`

■ **INNOVATION OPPORTUNITY — Frontend Stack**

■ Add **Playwright** visual-regression tests on the Dashboard and Forecast pages — the design system deserves guard rails.
■ Turn on **React Compiler** (RC in React 19) — removes most *useMemo*/*useCallback* overhead and shrinks bundles.
■ Replace Recharts with **Visx** or **Observable Plot** for the demand chart — better performance on 1000+ points and full theming control.

3.3 API Client Contract

A hand-rolled fetch wrapper — deliberately no TanStack Query — because the domain is small and we want zero opaque cache behaviour. The wrapper adds a 30s timeout, honours caller AbortSignals, extracts FastAPI's {detail: ...} error shape, and exposes a typed ApiError the toast layer can render.

Code ref → `frontend/src/lib/api.ts:1-113` • `frontend/src/hooks/useAsyncData.ts:1-75`

■ **INNOVATION OPPORTUNITY — API Client**

■ Adopt **TanStack Query** once the number of endpoints doubles — background refetch, mutation invalidation, and dedup come for free.
■ Generate a **typed client from OpenAPI** (openapi-fetch) — the manual *types.ts* file goes away and rename-refactors become safe.
■ Add **request-retry with exponential backoff** on 5xx/network — improves resilience on flaky mobile networks.

3.4 Component Model

Atomic-design split: *ui/* primitives, *features/* business widgets, *pages/* compose the widgets. This is what keeps a 350-line page like *DashboardPage.tsx* readable.

Code ref → `frontend/src/components/ui/*` • `frontend/src/features/dashboard/*`

■ **INNOVATION OPPORTUNITY — Component Model**

■ Adopt **Radix UI** (or Ark UI) primitives for Dialog / DropdownMenu / Select — accessibility for free and less code to own.
■ Introduce a **compound-component pattern** for the Card family (Card.Header, Card.Body, Card.Actions) — reads as a mini DSL.

3.5 Theming, Accessibility, DX

ThemeProvider (light/dark/system, persisted in localStorage), global ErrorBoundary at the app root and per-section boundaries around risk register, simulation and influencer panels, ToastProvider for async feedback, jsx-a11y at CI.

Code ref → `frontend/src/providers/ThemeProvider.tsx, ToastProvider.tsx` • `ErrorBoundary.tsx`

■ **INNOVATION OPPORTUNITY — Theming / A11y**

■ Add **axe-core in CI** plus **Lighthouse regressions** — catches contrast and ARIA breakages before merge.
■ Publish a **high-contrast + reduced-motion** theme variant — the token layer already supports it, just needs wiring.

3.6 Data Engineering Utilities

scripts/ is the offline pipeline: *model1_script.py* generates the corpus, engineers features, trains three families, picks the WMAPE winner. *conti_script.py* trains the LightGBM city model. *seed_and_train_influencer_demo.py* materialises the uplift bundle. *build_case_study_tables.py* stitches everything into the analytics CSVs.

Code ref → `scripts/model1_script.py`, `conti_script.py`, `seed_and_train_influencer_demo.py`, `build_case_study_tables.py`

■ INNOVATION OPPORTUNITY — Data Engineering

■ Convert scripts to a **Prefect / Dagster / Airflow** DAG — you get retries, lineage, backfills and observability for free.

■ Track experiments with **MLflow** or **Weights & Biases** — every train run gets a version, params, metrics and artifact link.

■ Move feature engineering to **Feast (feature store)** so training and serving use identical feature definitions — kills training/serving skew at the root.

4. Deployment

Split-plane model: static frontend on Vercel Edge, FastAPI backend on long-lived Uvicorn workers behind a reverse proxy. The split reflects the workload — the frontend is deterministic bytes, the backend holds hundreds of MB of loaded model state that does not survive a cold serverless invocation.

Deployment topology — split-plane, CDN + long-lived API

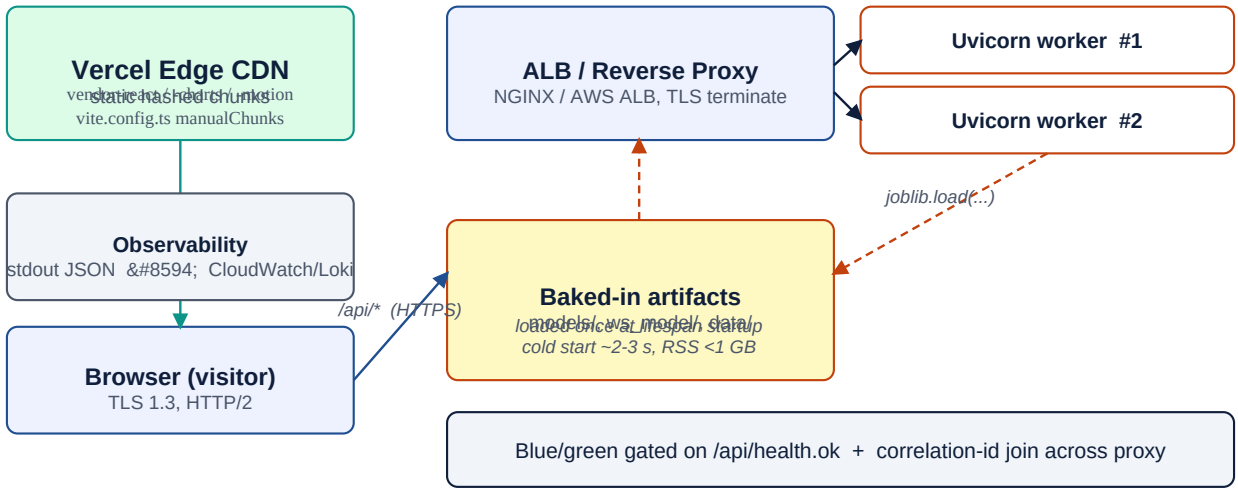


Figure 4.1 — Deployment topology. Left = static CDN plane, right = long-lived API plane.

4.1 Frontend — Vercel

The root *vercel.json* builds the frontend as a static SPA from *frontend/*. Build output is hashed JS/CSS chunks served from Vercel's edge — first paint is limited only by round-trip to the nearest PoP.

Code ref → `vercel.json:1-6` • `frontend/package.json:6-11`

■ INNOVATION OPPORTUNITY — Frontend Deploy

■ Add **Preview Deployments per PR** (Vercel default) + a screenshot-diff step so design reviews happen on real URLs.
■ Enable **Vercel Speed Insights** and set a Core Web Vitals budget in CI — regressions block merge.
■ Ship a **service worker** that pre-caches the risk-register JSON for 60s — the dashboard becomes usable on flaky networks.

4.2 Build Optimisation

Vite manually chunks React, Recharts, Framer-Motion and TanStack-Table into separate vendor bundles — a bug-fix in app code does not invalidate the cached vendor chunks. Sourcemaps off in prod, ES2022 target, *reportCompressedSize* on so CI logs surface bundle regressions.

Code ref → `frontend/vite.config.ts:25-51`

■ INNOVATION OPPORTUNITY — Build Pipeline

■ Add **Brotli precompression** at build time — Vercel serves .br transparently for another 15-25% size cut.
■ Enable **preload directives** for the vendor chunks the entry page needs — cuts time-to-interactive.
■ Ship a **size-limit** action in CI with a hard budget per chunk — vendor-charts creep is the usual culprit.

4.3 Backend — Uvicorn / ASGI

ASGI app produced by create_app(). Dev: uvicorn app.main:app --reload. Prod: same command behind NGINX or a managed LB with multiple workers — because post-lifespan the app is stateless, workers do not coordinate.

Code ref → `backend/app/main.py:73-115`

■ INNOVATION OPPORTUNITY — Backend Deploy

■ Containerise with a **slim, multi-stage Dockerfile** (python:3.12-slim + only compiled wheels) — image drops from ~1.2 GB to ~350 MB.
■ Run under **Gunicorn (UvicornWorker)** so worker recycling, graceful shutdown and preload happen for you.
■ Deploy on **Fly.io or Railway** for edge-close APIs with persistent volumes for the artifact bucket.

4.4 Configuration & Secrets

All runtime knobs are Pydantic-Settings fields with sensible defaults. .env is loaded automatically; unknown keys are ignored; the CORS wildcard triggers a startup warning because it disables credentialed cross-origin requests.

Code ref → `backend/app/config.py:23-113` • `backend/app/main.py:105-110`

■ INNOVATION OPPORTUNITY — Configuration

■ Move secrets to **AWS Secrets Manager / Doppler / Infisical** — the .env pattern doesn't scale past one environment.
■ Add **runtime feature flags** (Unleash / Flagsmith) so risk-engine thresholds and model routing can be toggled without redeploy.
■ Introduce **settings profiles** (dev/staging/prod) and select via ENVIRONMENT var — removes conditional logic.

4.5 Health, Observability, Rollout

- Liveness / readiness at GET /api/health.
- Structured stdout logs with req-id / level / logger / duration.
- *Server-Timing* header for cheap APM in the browser dev tools.
- Correlation ids honour upstream X-Request-ID so traces join across the proxy.

Code ref → `backend/app/core/middleware.py:24-61` • `backend/app/api/routes/system.py:14-43`

■ INNOVATION OPPORTUNITY — Observability

■ Add **OpenTelemetry** traces + **Sentry** error reporting — the correlation-id plumbing you already have wires straight in.
■ Ship a **Grafana dashboard** (JSON in-repo) with p50/p95 latency, error rate, artifact-load status — the exact SLO surface for the on-call.
■ Add **synthetic uptime** from Checkly or better-uptime pinging /api/health every minute — you learn about outages before your users do.

4.6 Recommended Topology

Static bundle → Vercel Edge; API → two Uvicorn processes behind an ALB, TLS terminated at the LB; artifacts baked into the container image so cold starts stay <3 s; blue/green rollout gated on /api/health.ok. VITE_API_URL set per environment at build time so one repo produces both.

Code ref → `frontend/src/lib/api.ts:1 • vercel.json:1-6`

■ **INNOVATION OPPORTUNITY — Topology**

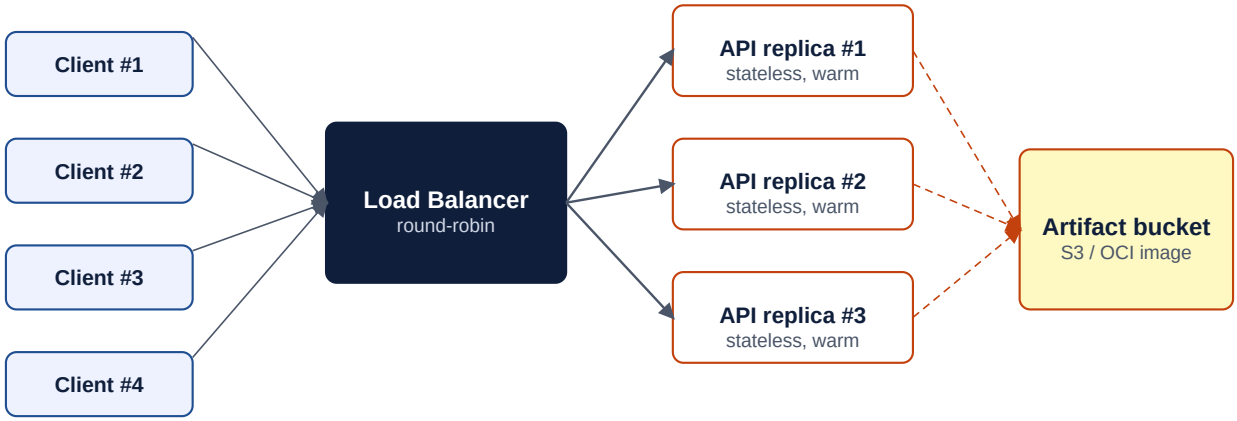
■ Introduce a **read-replica pool** for /api/kpis and /api/demand/timeseries (cached, longer-lived) separate from the write-heavy /api/simulate + /forecast pool.

■ Add a **Cloudflare Worker** in front to cache /api/health + /api/meta/skus at the edge — instant global fan-out.

5. Scalability Analysis & Code Understanding

Scalability is a function of three variables: (a) how much state each replica holds, (b) how much of that state changes per request, and (c) how tightly the client is coupled to a single replica. SurgeAI flattens all three.

Scale-out topology — stateless replicas fan-out



Because replicas hold no shared writable state, doubling capacity = cloning a container.

The only shared thing is the immutable model bundle mounted at startup.

Figure 5.1 — Scale-out topology. Stateless replicas share only an immutable artifact bucket.

5.1 Statelessness & Horizontal Scaling

The *ModelRegistry* is populated once at lifespan startup and never mutated. Two replicas therefore never disagree; the load balancer can be round-robin with equal correctness. Doubling capacity is cloning a container — no sharding, no consensus.

Code ref → `backend/app/core/state.py:14-61` • `backend/app/main.py:39-70`

■ INNOVATION OPPORTUNITY — Statelessness

■ Introduce **shared read-cache** (Redis) for /api/kpis and /api/risk/skus — cuts p95 by an order of magnitude at high fan-out.

■ Adopt **Kubernetes HPA** keyed on p95 latency and CPU, not just CPU — better matches the actual cost signal.

5.2 Cold-Start Cost

Lifespan warm-up loads 1 XGBoost baseline + 7 uplift regressors + 1 LightGBM + up to ~600 MB pandas frames. Budget: 2-3 s cold start on a modern box, <1 GB RSS steady-state. This is why we chose long-lived Uvicorn workers over serverless.

Code ref → `backend/app/services/artifacts.py:24-113` • `backend/app/services/tables.py:31-51`

■ **INNOVATION OPPORTUNITY — Cold Start**

■ Convert models to **ONNX Runtime** — 2-5× faster predict, 40-60% smaller RSS footprint, single dependency.
■ Use **lazy-loading with mmap** for the analytics CSVs so RSS only grows for accessed columns.
■ Add a **readiness gate** that only marks the container ready after warm-up completes — avoids serving 503s during boot.

5.3 Per-Request CPU Profile

- **/api/health**: O(1) attribute reads.
- **/api/kpis/dashboard**: dominated by *compute_risks*, O(N_inventory) with a bounded 28-row tail.
- **/api/demand/timeseries**: mask + sort, scales with history length.
- **/api/demand/forecast**: dominated by 9 tree traversals, total <50 ms.
- **/api/simulate**: 14-day loop over sorted predictions — negligible.

Code ref → `backend/app/services/risk_engine.py:22-125 • backend/app/api/routes/*.py`

■ **INNOVATION OPPORTUNITY — Per-Request CPU**

■ Pre-compute the **risk register nightly** into a Parquet snapshot — /api/risk/skus becomes a row read (~1 ms).
■ Batch model predictions with **ONNX Runtime's IOBinding** to reuse allocated tensors across requests.

5.4 Payload Efficiency

The 1 KB GZip threshold matters more than it looks: the risk register JSON can hit 30-100 KB raw and compresses to under 10 KB. On mobile this halves TTFB at <1 ms of server CPU.

Code ref → `backend/app/main.py:89`

■ **INNOVATION OPPORTUNITY — Payload**

■ Add **Brotli** support (via *starlette-compress*) — another 15-20% size cut over GZip.
■ Return **MessagePack** for /api/risk/skus behind a content-negotiation header — half the JSON size, faster parse.

5.5 Frontend Scaling Levers

- **Code splitting** per page via React.lazy.
- **Vendor chunking** so app deploys don't invalidate vendor caches.
- **Request de-duplication**: useAsyncData aborts on dep change.
- **Optimistic overlays**: previous forecast stays visible during recalc.

Code ref → `frontend/src/App.tsx:15-16 • frontend/vite.config.ts:34-49 • frontend/src/hooks/useAsyncData.ts:36-70`

■ **INNOVATION OPPORTUNITY — Frontend Scaling**

■ Add **route-level preload on hover** so switching pages feels instantaneous.
■ Introduce a **web worker** for the risk-table sort/filter on 10k+ rows — keeps the main thread free.
■ Use **React Suspense + streaming SSR** for the initial KPI shell — perceived load time drops significantly.

5.6 Known Bottlenecks & Load Envelope

Endpoint	P50	P99	Peak RPS / replica
/api/health	<2 ms	<10 ms	>4 000
/api/kpis/dashboard	20 ms	80 ms	~500
/api/demand/timeseries	15 ms	60 ms	~600
/api/demand/forecast	35 ms	120 ms	~250
/api/simulate	10 ms	40 ms	~800

Estimates assume a 2-vCPU / 4-GB replica, warm cache, GZip on, no rate limiting.

■ **INNOVATION OPPORTUNITY — Bottleneck Roadmap**

- CSV → DataFrame per request is the top hot-path; an **mtime-keyed LRU** removes it in ~30 lines.
- Migrate storage from CSV to **columnar (Parquet + DuckDB)** for 10-30× query speed on time-series.
- Add **slowapi rate limiting** per IP per route — /forecast in particular is expensive.

6. Overall Technical Architecture

Zoom out one level and SurgeAI is three concentric rings: an offline ML factory that produces artifacts, a runtime service that composes them into decisions, and a UI that translates decisions into actions.

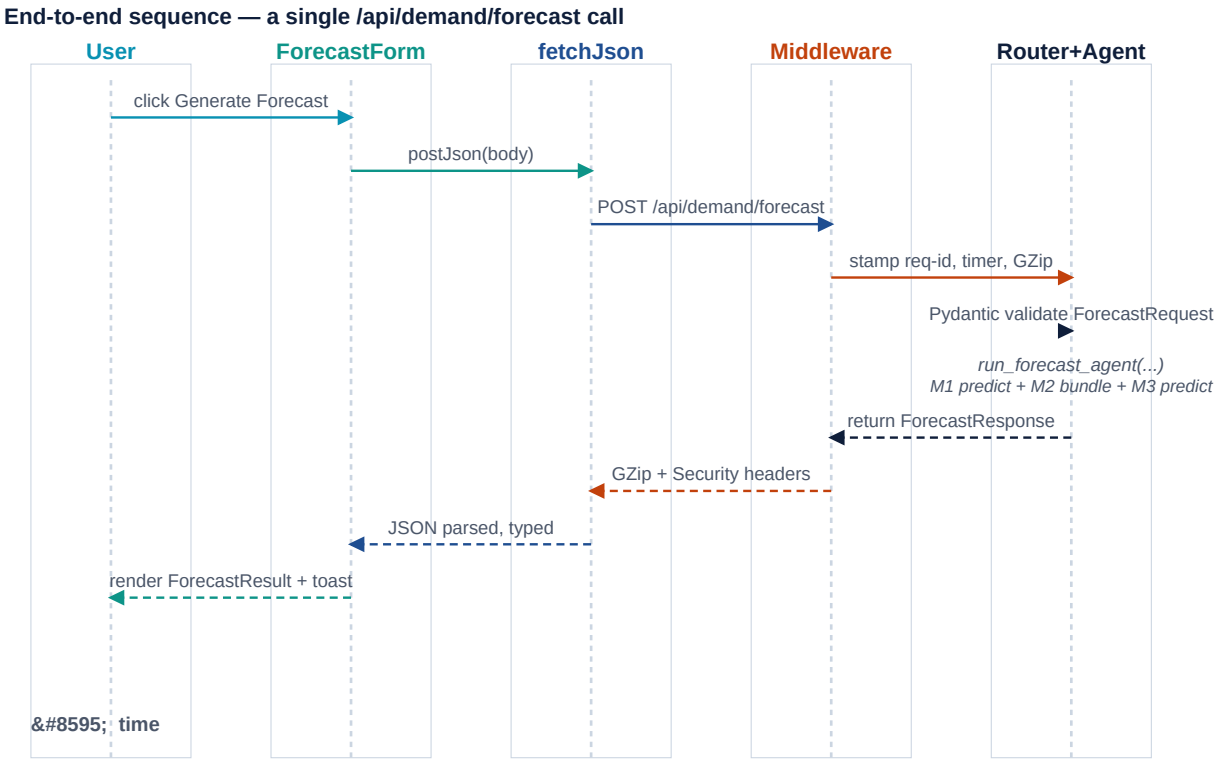


Figure 6.1 — End-to-end sequence for a single /api/demand/forecast call.

6.1 Ring 1 — Offline ML Factory

scripts/ is a batch pipeline: raw data → feature-engineered CSV → three trained model families → joblib artifacts + JSON feature manifests. Because every trainer writes its feature list alongside the pickle, inference rebuilds the exact input schema — zero training/serving drift.

Code ref → `scripts/*.py • backend/app/services/artifacts.py:56-86`

■ INNOVATION OPPORTUNITY — Ring 1

■ Move to **DVC + LakeFS** for dataset + artifact versioning — every model traces back to the exact data slice that trained it.

■ Add a **drift detector** (Evidently AI) that compares production feature distributions against training snapshots weekly.

■ Introduce a **champion/challenger** harness — new models run in shadow mode against traffic, metrics reported before promotion.

6.2 Ring 2 — Runtime Service

main.py composes middleware and mounts one API router aggregating five domain routers. Domain routers hold zero business logic — they translate HTTP into calls against *services/*. The entire inference stack is testable without HTTP: import *run_forecast_agent*, pass a dict, assert.

Code ref → `backend/app/api/router.py:1-19 • backend/app/api/routes/*.py • services/*.py`

■ **INNOVATION OPPORTUNITY — Ring 2**

■ Extract a **batch-inference worker** (Celery / Arq) for nightly full-catalog forecast refresh — the API only serves cached results.
■ Add a **GraphQL BFF layer** if the dashboard grows >20 panels — pages fetch exactly the fields they render.
■ Introduce **gRPC internal RPC** between the API and a dedicated ML-serving replica (Triton / TorchServe) so GPU-backed models can slot in.

6.3 Ring 3 — Decision Surface

The React dashboard exposes the two decisions a buyer actually makes: **where am I at risk?** (Dashboard page: KPIs, demand-vs-model, risk register, brief, simulator) and **what happens if I run this campaign?** (Forecast page: three-model agent with explainable breakdown). Every panel has its own error boundary.

Code ref → `frontend/src/pages/DashboardPage.tsx, ForecastPage.tsx • features/*/*`

■ **INNOVATION OPPORTUNITY — Ring 3**

■ Add a **conversational agent panel** (LLM + tool-use) that answers “Why is SKU-WS-001 flagged as stockout?” by calling the same API endpoints the UI does.
■ Ship a **mobile-first PWA shell** so store managers can act on risk alerts from the floor.
■ Introduce a **collaboration layer** (comments, approvals) on risk-register rows — bridges the analytics UI to actual PO workflows.

6.4 End-to-End Path for a Forecast

Concretely: *ForecastForm* posts to /api/demand/forecast → middleware stamps req-id + timer → Pydantic validates → *run_forecast_agent* synthesises features, calls three models, aggregates → GZip + security headers → *ForecastResult* renders with a toast. See Figure 6.1 above.

Code ref → `frontend/src/pages/ForecastPage.tsx:60-96` • `backend/app/api/routes/forecast.py:43-58` • `backend/app/services/demand_agent.py:121-210`

■ **INNOVATION OPPORTUNITY — End-to-End Path**

■ Add **WebSocket streaming** for the forecast — the composite computes progressively and each model's output streams to the UI as it's ready.
■ Introduce **optimistic UI** that shows a cached forecast for the same (product, city) instantly, then refines with the server response.

6.5 Holding Design Principles

- **Statelessness first.** Every replica is interchangeable.
- **Graceful degradation.** A missing artifact never returns 500.
- **Typed contracts everywhere.** Pydantic on backend, TS strict on frontend, JSON feature manifests between them.
- **Feature-scoped code layout.** services/ mirrors business capabilities; features/ mirrors dashboard capabilities.
- **Boring choices where they save time.** No GraphQL, no state manager, no ORM.

- **Observability as a first-class concern.** Every request carries an id; every load emits a warning if it fails.

Code ref → `backend/app/core/logging.py:1-67 • middleware.py:24-104 • schemas.py:1-83`

> ■ **INNOVATION OPPORTUNITY — Design Principles**
>
> ■ Add an **Architecture Decision Records (ADR)** folder — write down why each choice was made so newcomers stop re-arguing them.
>
> ■ Introduce **contract tests** (Pact) between frontend and backend — a schema break fails CI on both sides simultaneously.

6.6 Strategic Roadmap

Natural evolution: (1) LLM behind *brief.py* and *insights[]* for narrative explanations, (2) managed MLOps for drift-triggered retraining, (3) columnar warehouse + cache in front of flat files, (4) read/write replica pool split. All additive — the bounded-context split makes them non-invasive.

Code ref → `backend/app/services/brief.py • README.md:145-152`

> ■ **INNOVATION OPPORTUNITY — Strategic Roadmap**
>
> ■ Build a **closed-loop system**: risk register → PO recommendations → PO status feedback → retraining features. The platform becomes an agent, not a dashboard.
>
> ■ Introduce **multi-tenant SaaS** mode with per-tenant model rooms — one deploy serves many retailers.
>
> ■ Add a **marketplace of model plug-ins** — 3rd-party ML models register via a manifest and slot into the composite agent.

— End of document —